🤖 AI Anomaly Detection System - Complete Walkthrough

Overview

This blockchain-based supply chain project implements a **real-time AI-powered anomaly detection system** that monitors all supply chain activities and automatically flags suspicious or unusual patterns. The system uses **Isolation Forest** algorithm combined with **statistical analysis** to detect anomalies across four entity types: Raw Materials, Manufacturing, Distributor Inventory, and Retailer Inventory.

🏗️ System Architecture

```
graph TB
    A[Supply Chain Operations] -->|POST/PUT/PATCH| B[API Routes]
    B --> C[Anomaly Monitor Middleware]
    C --> D[Anomaly Detector Service]
    D --> E[Isolation Forest Model]
    D --> F[Feature Extraction]
    E --> G{Anomaly Score > 0.5?}
    G -->|Yes| H[Create Alert]
    G -->|No| I[Save Normal Data]
    H --> J[MongoDB - Anomaly Alerts]
    I --> K[MongoDB - Anomaly Data]
    H --> L[WebSocket Notification]
    L --> M[Frontend Dashboard]
    J --> M
```

📊 Core Components

1. Data Models

AnomalyData Model

Stores historical data used for training the anomaly detection models.

Key Fields:

- `entityType` : Type of entity (raw_material, manufacturing, distributor, retailer)
- `features` : Extracted numerical features (quantity, price, time metrics)
- `metrics` : Calculated z-scores and anomaly scores
- `isAnomaly` : Boolean flag indicating if this was flagged as anomalous

Location: [anomalyDataModel.js]

AnomalyAlert Model

Stores alerts generated when anomalies are detected.

Key Fields:

- `alertId` : Unique identifier (e.g., ALERT-1732649123456-abc123)
- `severity` : low, medium, high, or critical
- `anomalyType` : Type of anomaly (quantity_anomaly, price_anomaly, time_anomaly, etc.)
- `status` : new, acknowledged, resolved, or false_positive
- `anomalyScore` : Numerical score from 0-1 (higher = more anomalous)
- `metadata` : Contains features and sanitized entity data

Location: anomalyAlertModel.js

---

2. Anomaly Detection Service

The core AI engine that powers the anomaly detection system.

Location: anomalyDetector.js

Key Components:

Statistical Helper Functions

```
calculateMean(values)      // Calculates average
calculateStdDev(values)    // Calculates standard deviation
calculateZScore(value, mean, stdDev)  // Calculates z-score
```

Isolation Forest Algorithm

A simplified implementation of the Isolation Forest algorithm for anomaly detection:

- **Training (`fit` method):**
  - Takes historical data as input
  - Calculates mean and standard deviation for each feature
  - Builds a baseline for normal behavior
- **Prediction (`predict` method):**
  - Compares new data against the baseline
  - Calculates z-scores for each feature
  - Converts z-scores to anomaly scores (0-1 scale)
  - Z-score > 5 = maximum anomaly (score = 1.0)

AnomalyDetector Class

Initialization:

```
async initialize() {
  // Loads last 1000 non-anomalous records from database
  // Groups data by entity type
  // Trains separate models for each entity type
}
```

Feature Extraction:

```
extractFeatures(entity, entityType) {
  // Extracts numerical features from entities:
  // - quantity
```

```
  // - pricePerUnit
  // - timeSinceLastUpdate (in hours)
  // - dayOfWeek, hourOfDay, isWeekend
}
```

Anomaly Detection:

```
async detectAnomaly(entity, entityType) {
  1. Extract features from entity
  2. Get anomaly score from model
  3. Determine severity based on score:
     - score > 0.8 → critical
     - score > 0.6 → high
     - score > 0.4 → medium
     - otherwise → low
  4. If score > 0.5, create alert
  5. Save data for future training
}
```

Batch Scanning:

```
async scanAll() {
  // Scans last 100 records from each entity type
  // Runs anomaly detection on each
  // Returns summary of detected anomalies
}
```

---

3. Middleware Integration

Location: [anomalyMonitor.js]

The `anomalyMonitor` middleware intercepts successful API responses and triggers anomaly detection asynchronously.

How it works:

1. Wraps around route handlers
2. Intercepts `res.json()` calls
3. For POST/PUT/PATCH requests with 2xx status codes:
   - Extracts entity from response
   - Runs anomaly detection in background (non-blocking)
   - Emits WebSocket event if anomaly detected
4. Returns response immediately (doesn't block user)

Example Usage:

```
router.post('/', verifyToken, authorize(['MAN']),
  anomalyMonitor('manufacturing'),
  async (req, res) => {
    // Route handler code
```

```
  }
);
```

Integrated Routes:

- Raw Material routes (POST, PUT, PATCH)
- Manufacturing routes (POST, PATCH, dispatch)
- Distributor Inventory routes (POST, PATCH)
- Retailer Inventory routes (POST, PATCH)

---

4. API Endpoints

Location: [anomalyRoutes.js]

Available Endpoints:

Method	Endpoint	Description
GET	`/api/anomalies`	Fetch all alerts with filtering
GET	`/api/anomalies/:id`	Get specific alert by ID
PATCH	`/api/anomalies/:id/acknowledge`	Mark alert as acknowledged
PATCH	`/api/anomalies/:id/resolve`	Resolve alert or mark as false positive
POST	`/api/anomalies/scan`	Trigger manual scan (ADMIN only)
GET	`/api/anomalies/stats/summary`	Get statistics and analytics

Query Parameters for GET /api/anomalies:

- `severity` : Filter by severity (critical, high, medium, low)
- `status` : Filter by status (new, acknowledged, resolved, false_positive)
- `affectedEntity` : Filter by entity type
- `startDate` , `endDate` : Date range filtering
- `limit` , `skip` : Pagination

---

5. Server Integration

Location: [server.js]

The server initializes the anomaly detection system on startup:

```
// Initialize on server start
anomalyDetector.initialize()

// Periodic scanning every 6 hours
setInterval(async () => {
  await anomalyDetector.scanAll();
}, 6 * 60 * 60 * 1000);

// WebSocket support for real-time notifications
```

```
const io = new Server(httpServer);
app.set('io', io);
```

🎨 Frontend Components

1. Anomaly Alerts Page

Location: [AnomalyAlerts.js]

Features:

- **Real-time updates** via WebSocket
- **Filtering** by severity, status, and entity type
- **Alert management** (acknowledge, resolve, mark as false positive)
- **Desktop notifications** for new anomalies
- **Detailed modal view** for each alert

Key Functionality:

```
// WebSocket connection for real-time alerts
const socket = io('http://localhost:5002');
socket.on('anomaly_detected', (data) => {
  setAlerts(prev => [data.alert, ...prev]);
  showNotification(data.alert);
});

// Alert actions
acknowledgeAlert(alertId)  // Changes status to 'acknowledged'
resolveAlert(alertId, notes, isFalsePositive)  // Resolves or marks false positive
```

2. Anomaly Dashboard

Location: [AnomalyDashboard.js]

Features:

- **Summary statistics** (total, by severity, by status)
- **Time range selection** (24 hours, 7 days, 30 days)
- **Visual breakdowns** of severity and anomaly types
- **Progress bars** showing distribution

Displays:

- Total alerts count
- Breakdown by severity (critical, high, medium, low)
- Status overview (new, acknowledged, resolved)
- Anomaly type distribution

🔄 Complete Workflow

Real-Time Detection Flow

1. **User Action:** A manufacturer creates a new product entry

```
POST /api/manufacturing
```

2. **Route Handler:** Processes the request with `anomalyMonitor` middleware

```
router.post('/', verifyToken, authorize(['MAN']),
  anomalyMonitor('manufacturing'),
  async (req, res) => { ... }
);
```

3. **Response Interception:** Middleware intercepts successful response

```
res.json = function(data) {
  // Extract entity from response
  // Run anomaly detection asynchronously
}
```

4. **Feature Extraction:** System extracts features

```
{
  quantity: 500,
  pricePerUnit: 25.50,
  timeSinceLastUpdate: 2.5,
  dayOfWeek: 2,
  hourOfDay: 14,
  isWeekend: false
}
```

5. **Anomaly Scoring:** Isolation Forest calculates score

```
// Compare against baseline
// Calculate z-scores for each feature
// Aggregate into overall anomaly score
anomalyScore = 0.72  // Example: High anomaly
```

6. **Alert Creation:** If score > 0.5, create alert

```
{
  alertId: "ALERT-1732649123456-abc123",
  severity: "high",
  anomalyType: "quantity_anomaly",
  affectedEntity: "manufacturing",
  anomalyScore: 0.72,
  description: "Anomaly detected in manufacturing (score: 72.0%). Quantity: 500.
Price per unit: 25.5."
}
```

7. **Real-time Notification:** WebSocket broadcasts to connected clients

```
io.emit('anomaly_detected', {
  alert: alertData,
  timestamp: new Date()
});
```

8. **Frontend Update:** Dashboard receives and displays alert

```
socket.on('anomaly_detected', (data) => {
  // Add to alerts list
  // Show browser notification
});
```

Batch Scanning Flow

1. **Trigger:** Manual scan or scheduled scan (every 6 hours)

```
POST /api/anomalies/scan  (Manual)
setInterval(() => scanAll(), 6hrs)  (Scheduled)
```

2. **Data Collection:** Fetch last 100 records from each entity type

```
const rawMaterials = await RawMaterial.find().limit(100);
const manufacturing = await Manufacturing.find().limit(100);
const distributorItems = await DistributorInventory.find().limit(100);
const retailerItems = await RetailerInventory.find().limit(100);
```

3. **Batch Processing:** Run detection on each record

```
for (const item of manufacturing) {
  const result = await detectAnomaly(item, 'manufacturing');
  if (result.isAnomaly) results.manufacturing.push(result);
}
```

4. **Results Summary:** Return aggregated results

```
{
  raw_material: 2,
  manufacturing: 5,
  distributor: 1,
  retailer: 3,
  total: 11
}
```

🎯 Anomaly Types Detected

Type	Description	Detection Criteria

quantity_anomaly	Unusual quantity values	Quantity > 1000 or high z-score
price_anomaly	Unusual pricing	Price per unit > 10000 or high z-score
time_anomaly	Unusual timing patterns	Time since update > 720 hours (30 days)
pattern_mismatch	General statistical outlier	High overall anomaly score

---

📈 Severity Levels

Severity	Anomaly Score Range	Action Required
Critical	> 0.8	Immediate investigation
High	0.6 - 0.8	Prompt review
Medium	0.4 - 0.6	Monitor closely
Low	< 0.4	Informational

🔧 Key Features

✅ Real-time Detection

- Monitors all POST/PUT/PATCH operations
- Non-blocking asynchronous processing
- WebSocket notifications for instant alerts

✅ Self-Learning System

- Continuously learns from historical data
- Adapts baseline as new normal data arrives
- Separate models for each entity type

✅ Comprehensive Analytics

- Statistics by severity, status, and type
- Time-based filtering (24h, 7d, 30d)
- Visual dashboards with progress bars

✅ Alert Management

- Acknowledge alerts
- Resolve with notes
- Mark false positives
- Track who resolved what and when

✅ Scheduled Scanning

- Automatic scans every 6 hours
- Manual scan capability for admins
- Batch processing of recent records

🚀 How to Use

For Users:

1. **View Alerts:** Navigate to Anomaly Alerts page
2. **Filter:** Use dropdowns to filter by severity, status, or entity
3. **Investigate:** Click on alert to see detailed information
4. **Take Action:** Acknowledge, resolve, or mark as false positive

For Administrators:

1. **Monitor Dashboard:** Check Anomaly Dashboard for overview
2. **Manual Scan:** Trigger manual scan via API or admin panel
3. **Review Statistics:** Analyze trends over different time periods
4. **Manage Alerts:** Resolve critical alerts with detailed notes

---

🔍 Example Scenario

Scenario: A distributor adds an unusually large shipment

```
// 1. Distributor creates inventory entry
POST /api/distributor-inventory
{
  quantity: 10000,  // Unusually high
  shippingDetails: { cost: 50000 }
}

// 2. Anomaly detector extracts features
features = {
  quantity: 10000,
  pricePerUnit: 5,
  timeSinceLastUpdate: 0.5,
  dayOfWeek: 3,
  hourOfDay: 10
}

// 3. Model calculates anomaly score
// Quantity z-score = 8.5 (very high!)
// Overall anomaly score = 0.85

// 4. Alert created
{
  severity: "critical",
  anomalyType: "quantity_anomaly",
  description: "Anomaly detected in distributor (score: 85.0%). Quantity: 10000.",
  anomalyScore: 0.85
}

// 5. Admin receives real-time notification
// 6. Admin investigates and either:
//    - Resolves as legitimate bulk order
//    - Flags as potential fraud
```

📝 Summary

The AI Anomaly Detection System provides:

- **Automated monitoring** of all supply chain activities
- **Real-time alerts** for suspicious patterns
- **Statistical analysis** using Isolation Forest algorithm
- **Self-learning capabilities** that improve over time
- **Comprehensive dashboards** for visualization
- **Alert management** workflow for investigation and resolution

This system helps maintain supply chain integrity by automatically detecting and flagging unusual activities that could indicate errors, fraud, or other issues requiring attention.